Bronze, Silver, Gold

An Opinionated Guide to the Medallion Architecture for Microsoft Fabric



Bas Land · ThatFabricGuy · Microsoft Data Platform MVP

Retro Data: Live · 26 September 2026

Your silver layer is doing your gold layer's job.

That's why nobody trusts your numbers. And that's why every change request takes three weeks.

I've built and inherited a lot of these platforms.

I've also rebuilt several of them. It was almost always the same reason.

The deal

- There is no “it depends” in this session.
- I'm going to tell you exactly what goes in each layer, and exactly why.
- You are going to disagree with at least one of them.
- I'd rather you disagree with something specific than nod along to something vague.
- By the end of the hour you'll have one rule you can apply to your own platform.
- Two worked examples from a real Fabric platform. Slides and code afterwards.

Bas Land

- Data consultant since 2013
- I’ve done Power Pivot, SQL Server, Power BI, Azure SQL, and now Fabric
- Partner @ Kimura, a Data & AI software and consultancy company
- Microsoft Dataplatform MVP
- Brazilian jiu-jitsu brown belt, weight lifter, runner
- Husband and father



Nobody agrees. That is not your fault.

The confusion isn't yours. The sources contradict each other, and one of them contradicts itself.

What the docs say about silver

Microsoft Learn: “Fix errors, standardize formats, and remove duplicates.”

Databricks: “… where you perform data cleansing, deduplication, and normalization.”

Microsoft's own worked examples:

- Standardize date formats across all sources
- Convert all currency to USD
- Remove test transactions
- Match customer records across systems

… and about gold

> Microsoft: “Organize for reports and dashboards.”

> Databricks: “Consists of aggregated data tailored for analytics and reporting.”

Hold on to that word.

Everything that has gone wrong with your silver layer follows from it.

Two of these four are business rules

Standardize date formats across sources

Format. Not a decision.

Silver

Convert all currency to USD

The rate changes. So does the reporting currency.

Business rule

Remove test transactions

Somebody decided what counts as a test.

Business rule

Match customer records across systems

That's Kimball. That's gold.

Conformed dimension

Define gold as aggregates, and business logic has nowhere to live.

Aggregates are an output, not a place to put logic. So the logic goes into silver, the one layer that must never hold it. The vendors created the vacuum. Silver filled it.

My gold layer is a dimensional model at atomic grain, not a pile of aggregates.

Once gold is a real modelling layer, silver doesn't have to be one. That's the whole argument.

Business rules change. History doesn't.

So business rules never live in a layer that stores history.

Finance redefines “active customer”

If the rule lives in gold

- Change one notebook.
- Rerun it.
- Twenty minutes.
- Not one byte reloaded from the source. Not one row of history rewritten.

If the rule lives in silver

- Option A: rewrite history
- The past now says something it never said. Your audit trail is fiction.
- Option B: keep both rules forever
- CustomerActive_v1. CustomerActive_v2. In three years nobody knows which one the report uses.

Whose language is this column in?

Source's language → bronze and silver. Business's language → gold.

The moment you translate, you are in gold.

The test, applied

Cast CREDIT_LIMIT from string to decimal	Bronze	The source always meant a number.
Deduplicate on the source's business key	Silver	The source's own key.
Track when a row changed (Sys_ValidFrom)	Silver	The history you're protecting.
Rename NO_ to CustomerNumber	Gold	Translation into business language.
Filter out test customers	Gold	A business rule. It will change.
Convert all currency to EUR	Gold	A rate that changes.
Match customers across CRM and ERP	Gold	A conformed dimension. Kimball.

Three layers, three questions



A layer that answers two of those questions answers neither one well.

Bronze

The arrival record. Nothing more, and nothing less.



Bronze rules (1–3)

1

The raw file lands in Files/ first. Always.

Including from a database. The extract becomes a file before it becomes a table. If ingest breaks at 3am you want the payload, not a stack trace.

2

Bronze Delta tables mirror source objects 1:1.

Same grain, same names. No unions, no splits, no renames. However ugly.

3

Typing happens here, not in silver.

Source-native types on the way into the Delta table. Casting isn't a business decision, so do it as early as you can do it reliably.

Bronze rules (4–6)

4

Append-only. Nothing is ever updated or deleted.

Bronze is a log of arrivals, not a picture of the source.

5

Raw files are kept 30 days, then aged out.

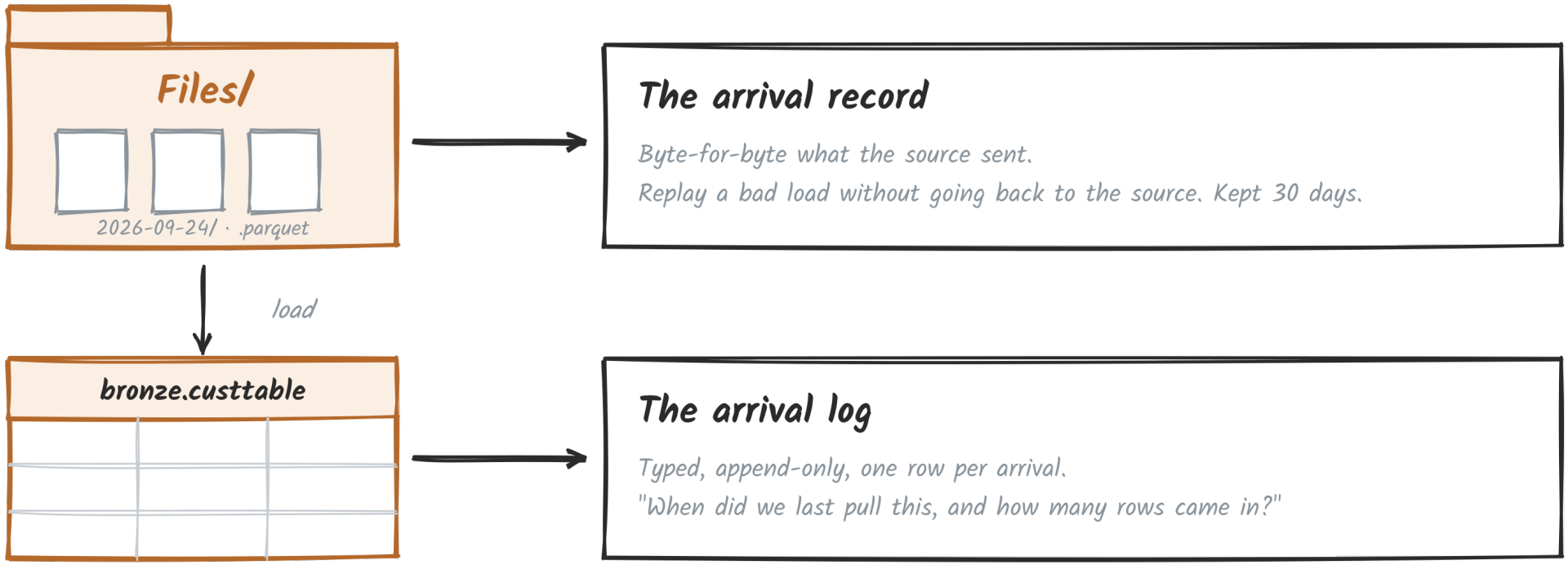
A number, not a policy discussion. Yours may be 90, but it is a number.

6

Nobody builds anything on bronze.

Not a report, not a semantic model, not a “quick look.” If someone can query it, someone will report on it.

Bronze does two jobs, not one



So yes, bronze has Delta tables. You need the file AND the table.

Silver

The mirror. This is the part you're going to argue with.



What silver is

- A faithful, versioned, schema-managed mirror of the source.
- One table per source object. Source column names, exactly as the source spells them.
- Deduplicated on the source's business key. The source's own ordering picks the winner, not me.
- Schema evolution is automated. A new column appears at source, silver picks it up and keeps running.
- Full history in columns. Every version of every row.
- Enriched with nothing except system fields.

The system columns, in full

Sys_ValidFrom	When this version became true
Sys_ValidTo	When it stopped. 9999-12-31 on the open row, never null
Sys_IsActive	Convenience flag for the current version
Sys_IsDeleted	The source stopped sending this row
Sys_Timestamp	Human-readable load id, yyyymmddhhmmss

Not Delta time travel. That is a file-retention feature with a vacuum horizon. You can't join to it, you can't govern it, and it vanishes when someone optimises the table.

Real history is columns.

What silver refuses

- No joins across source systems.
- No joins within a source system either. No flattening header and lines.
- No calculated columns.
- No aggregations.
- No filtering that encodes a business decision.
- No renaming. NO_ stays NO_.
- No PII masking. The source shouldn't send it; what lands is masked in gold.

Yes, my silver layer is ugly.

It's a mirror. A mirror that flatters you is broken. Nobody enjoys querying it, and nobody is supposed to. It has exactly one consumer, and that consumer is gold.

Objection 1

> “Gold becomes a monolith. All the complexity ends up in one layer.”

- **You haven't reduced complexity by spreading it. You've hidden it.**

  The same twelve transformations exist either way. The only question is whether you can find them.

- **Gold is forty small notebooks.**

  Every one has the same four-cell skeleton, and every one reruns on its own. That is the opposite of how a monolith fails.

- **“Which layer is it in?” is the most expensive question in data engineering.**

  This architecture doesn't have that question. There is one place to look, and it is always the same place.

The honest concession: this only works if you're disciplined about conventions. Same notebook structure every time, same naming, code review. If your team won't do that, gold does rot. It just rots visibly.

Objection 2

> “Silver is a wasted layer. Just query bronze.”

- **Bronze is a log of loads. Silver is a picture of entities.**

  Query bronze and every consumer writes the same window function over load timestamps, and gets it subtly wrong in a different way each time.

- **Deletes.**

  A full-load source expresses a delete by absence. Detecting absence in an append-only log is real work. Silver does it once and exposes Sys_IsDeleted.

- **Point-in-time becomes a where clause for everyone downstream.**

  Instead of a technique each consumer has to reinvent, and get slightly differently wrong.

The honest concession: one source, full loads, nobody ever asks what last quarter looked like. Your silver is thin and you could probably skip it. I've never worked on that platform, but it exists.

Objection 3

> “Microsoft and Databricks say otherwise.”

- **Agreement on bronze. Half agreement on silver.**

  Deduplication and normalisation, yes. Currency conversion and removing test transactions, no. Those are business rules.

- **The disagreement that matters is gold.**

  Define gold as aggregates and business logic has nowhere to live but silver. Define gold as a dimensional model and silver is free to stay dumb.

- **The docs describe a lakehouse without a modelling layer.**

  I build one with a modelling layer, so my silver does less.

The vendor definitions are naming conventions, not specifications. They were written for a world where the lakehouse was the warehouse and nothing downstream did dimensional modelling.

One customer, bronze to silver

What the data looks like at each step. Same customer all the way through.

bronze.custtable: every load, every time

NO_	NAME	CREDIT_LIMIT	loaded	
C10042	Acme Trading BV	10000.00	2026-01-01	
C10042	Acme Trading BV	10000.00	2026-02-01	nothing changed
C10042	Acme Trading BV	25000.00	2026-03-01	
C10042	Acme Trading BV	40000.00	2026-08-12	

Same customer, four times. The February load changed nothing, and bronze kept it anyway.
It's a log of arrivals. That is its whole job.

silver.custtable: one row per version

NO_	CREDIT_LIMIT	Sys_ValidFrom	Sys_ValidTo	Sys_IsActive
C10042	10000.00	2026-01-01	2026-03-01	0
C10042	25000.00	2026-03-01	2026-08-12	0
C10042	40000.00	2026-08-12	9999-12-31	1

Four loads became three versions, because February changed nothing.

And it still says NO_ and CREDIT_LIMIT. I didn't make it prettier, on purpose.

Time travel is a where clause

What was Acme's credit limit on 15 May?

```
select  no_, credit_limit
from    silver.custtable
where   no_ = 'C10042'
  and   sys_validfrom <= '2026-05-15'
  and   '2026-05-15'  <  sys_validto
```

NO_	CREDIT_LIMIT
C10042	25000.00

No vacuum horizon. No version numbers.

Anyone downstream can run it, now or in five years.

Gold

The only layer where a business term is allowed to appear.



Gold rules (1–4)

1

Kimball, at atomic grain. Not aggregates.

Aggregates can exist, but they're not the only reason for Gold.

2

One notebook per gold table.

Fixed structure every time: parameters → schema → mapping → upsert. Same skeleton, forty times.

3

Surrogate keys are hashes, not lookups: SHA2(CONCAT_WS('~', <natural keys>), 256)

No key-management table, no sequence, no lookup join. Facts build in parallel with their dimensions, because the key is deterministic.

4

SCD2 only where the business asked for it.

Type 1 is the default. Type 2 is a request, not a reflex.

Gold rules (5–7)

5

No null foreign keys, ever. Unknown rows instead.

A fact that can't find its dimension points at a real row that says so. Nulls in a star schema are how you get two different totals for the same measure.

6

Gold is derivable.

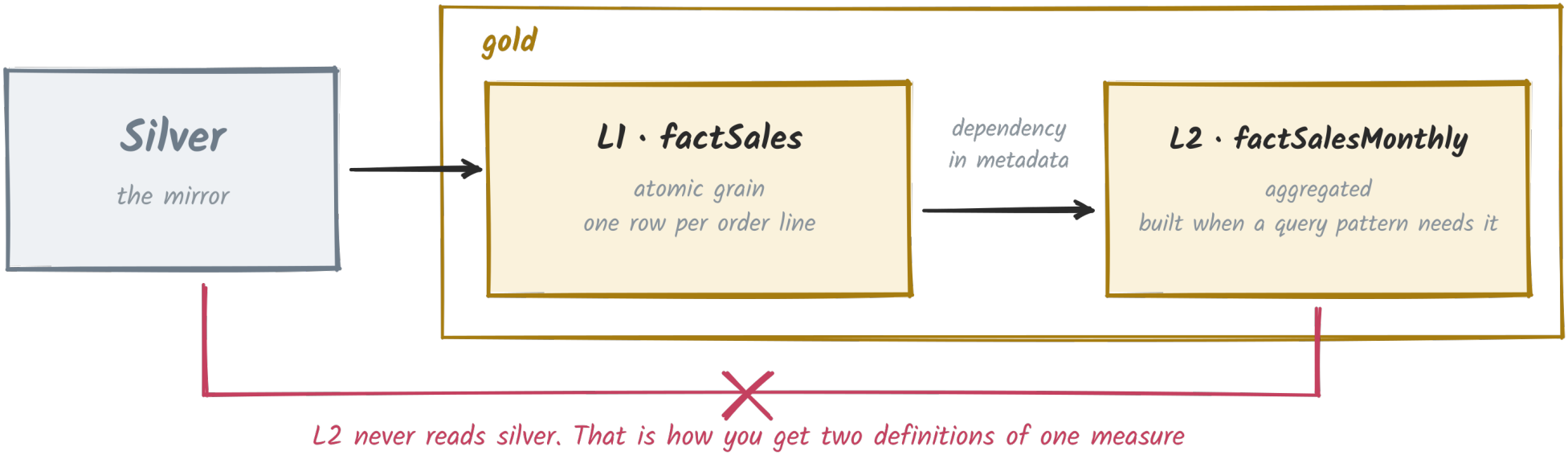
If there is anything in gold you could not reconstruct from silver, your history is in the wrong layer. A diagnostic, not a boast. See the next slide.

7

All the ugliness from silver lands here, in one visible place.

The renames, the case decodes, the coalesce(..., 'Unknown'). Complexity you can see is not a monolith.

Atomic grain, and what sits on top



"Gold is not aggregates" means aggregates are never the foundation.

Gold is derivable, not disposable

Type 1 gold

- Truncate. Rerun. Same result.
- Minutes.
- This is the easy case, and it's most of your tables.

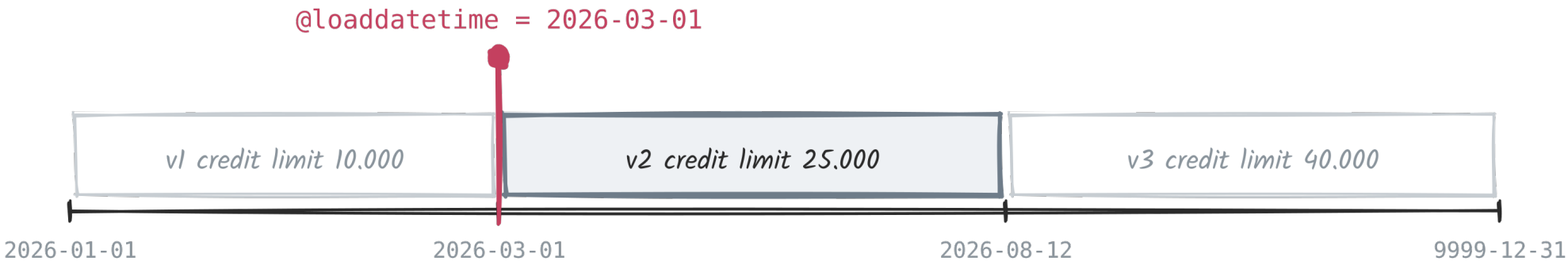
Type 2 gold

- Replay silver's load timestamps, in order.
- One run per historical load. Slower, and a hassle.
- But nothing is lost, because the history it needs lives in silver rather than gold.

Don't ask “can I truncate gold.”

Ask “is there anything in gold I couldn't reconstruct from silver?” If yes, your history is in the wrong layer.

Point-in-time is a parameter



`between Sys_ValidFrom and Sys_ValidTo`

✗ *Inclusive both ends. Matches v1 AND v2.*
The row is counted twice, silently.

`>= Sys_ValidFrom and < Sys_ValidTo`

✓ *Half-open. Matches v2, and only v2.*
Needs v1.ValidTo = v2.ValidFrom, 9999 on the open row.

When the same cleaning is needed twice

Used once

A cell in the gold notebook that needs it. It lives next to the thing it serves.

Used in several places

A small pre_ table, prepared once in gold and joined against by whoever needs it.

The guardrail: a pre_ table is never read by a report or a semantic model. It exists to be joined by other gold notebooks.

"Yes, that is a staging step with a prefix on it."

Same complexity, different location. I'd rather name it and put it where I can see it than hide it in the layer that holds my history.

From silver to a star schema

What one gold notebook looks like, and what happens when the business changes its mind.

One notebook, four cells

1 parameters

@loaddatetime, and which columns are SCD2

2 schema

Target columns and types, declared once

3 mapping

Silver in, business names out. The logic lives here.

4 upsert

Merge on the hashed key

```
-- cell 3: mapping
select   sha2(concat_ws('~', c.no_), 256)    as CustomerKey
      ,  c.no_                               as CustomerNumber
      ,  coalesce(c.name, 'Unknown')         as CustomerName
      ,  case when c.blocked = 0 then 'Active'
              else 'Blocked' end             as CustomerStatus
      ,  coalesce(sp.name, 'Unknown')        as AccountManager
from     silver.custtable c
left join silver.salesperson sp
        on sp.code = c.salesperson_code
       and sp.sys_validfrom <= @loaddatetime and @loaddatetime < sp.sys_validto
where    c.sys_validfrom <= @loaddatetime and @loaddatetime < c.sys_validto
```

Who owned it then, who owns it now

CustomerNumber	AccountManager	AccountManagerActual	ValidFrom	ValidTo
C10042	Sanne de Vries	Joris Bakker	2026-01-01	2026-03-01
C10042	Joris Bakker	Joris Bakker	2026-03-01	9999-12-31

```
scd_columns = ["AccountManager"]
```

AccountManager keeps history.

Who owned the customer when the sale happened.

AccountManagerActual is overwritten.

Who owns it today. The business asked both questions, so it gets both columns.

The key is calculated

```
-- nb_factSales, cell 3 (excerpt)
select  sha2(concat_ws('~', l.document_no_, l.line_no_), 256)                 as SalesLineKey
     ,  sha2(concat_ws('~', coalesce(h.sell_to_customer_no_, 'Unknown')), 256) as CustomerKey
     ,  sha2(concat_ws('~', coalesce(l.no_, 'Unknown')), 256)                 as ProductKey
     ,  l.quantity                                                            as Quantity
     ,  l.amount_lcy                                                          as Amount
from    silver.salesline l
join    silver.salesheader h on h.no_ = l.document_no_
-- plus the same @loaddatetime window on both tables
```

Same formula as dimCustomer

The keys match without a lookup, so this fact builds in parallel with its dimensions.

A customer that isn't there?

It hashes to the Unknown row in dimCustomer. No null foreign keys, as promised.

Finance changes its mind

New rule: customers blocked only for shipping still count as active.

```
-- nb_dimCustomer, cell 3
- ,  case when c.blocked = 0       then 'Active' else 'Blocked' end  as CustomerStatus
+ ,  case when c.blocked in (0, 1) then 'Active' else 'Blocked' end  as CustomerStatus
```

blocked: 0 = not blocked, 1 = ship, 2 = invoice, 3 = all

Active customers, before

812



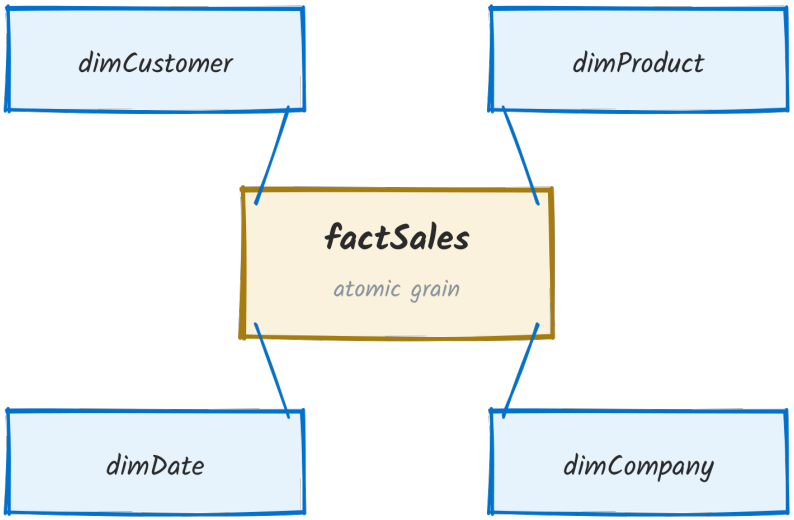
Active customers, after one rerun

947

I changed one business rule.

I didn't reload a byte from the source. I didn't touch bronze or silver. I didn't rewrite a row of history. One notebook, one rerun. If finance changes its mind back next quarter, it's the same twenty minutes.

What this unlocks



1

Semantic models that behave

Direct Lake over a real star. DAX that isn't compensating for a bad model.

2

AI agents that actually answer

A star schema with business-named dimensions is the vocabulary an agent needs.

3

Auditability

One number is wrong. There is exactly one notebook to open.

4

Change velocity

New source? It plugs into conformed dimensions you already have.

The dashboard is no longer the only thing reading your gold layer.

That, more than anything, is why it is worth building properly in 2026.

Bronze records what arrived. Silver tracks what changed. Gold explains.

Three different jobs. Keep them separate and the platform stays cheap to change. Everything you are going to be asked to build next assumes you did.



Questions

Slides and code samples

- The slides and code samples for this session will be posted to my public Github repo:
  - https://github.com/basland/presentations/
- Also check my YouTube and blog for content like this:
  - YouTube.com/@ThatFabricGuy
  - https://thatfabricguy.com

Connect with me

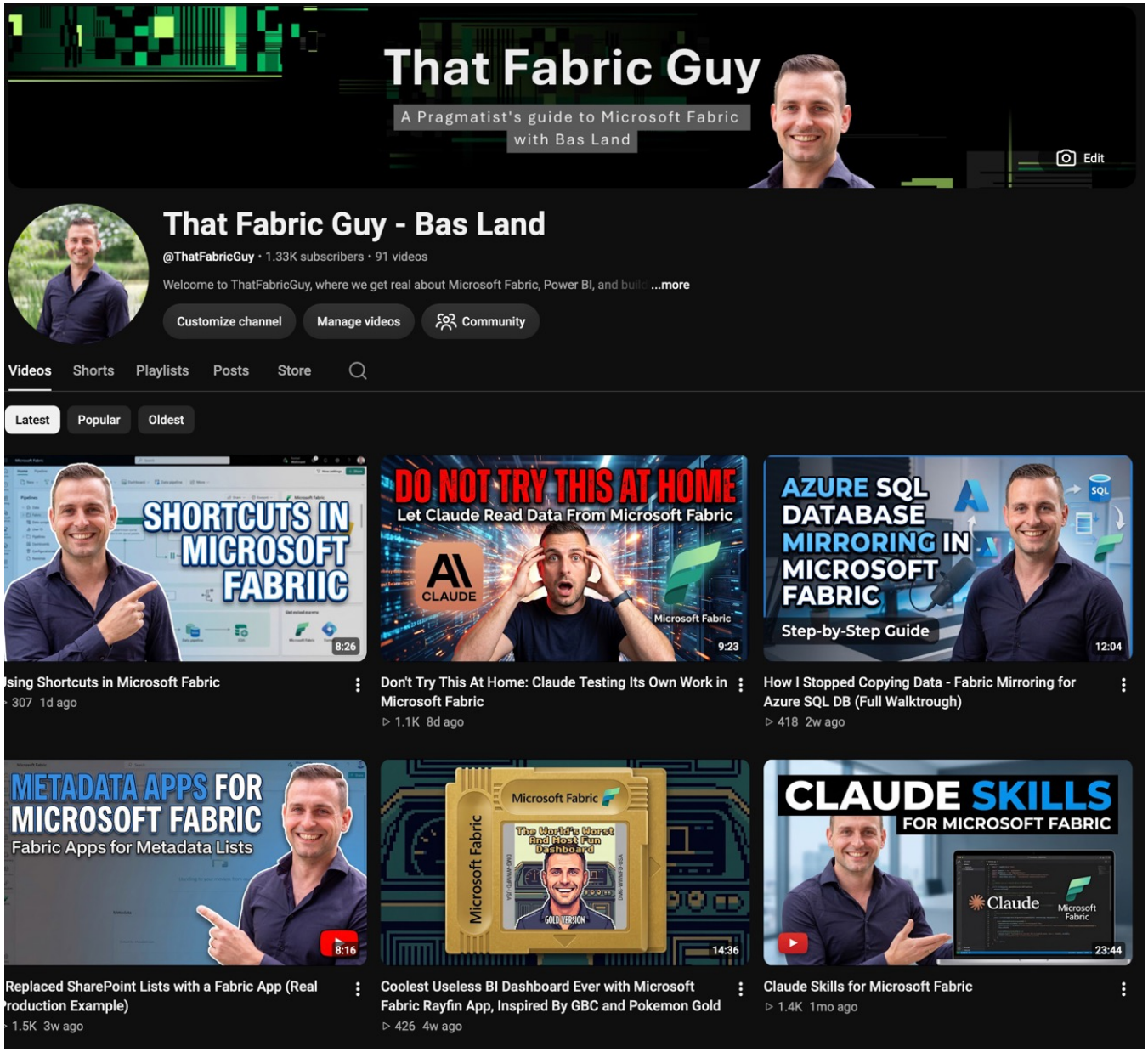
LinkedIn



YouTube



Follow my YouTube.
Seriously!



Thank you!